Programación declarativa @ URJC

Programación funcional

Curso 24-25, convocatoria ordinaria (28 de octubre de 2024)

Campus de Vicálvaro

La duración del examen es de 1h:30m.

Preámbulo

In [ ]:
```
import $ivy.`org.scalatest::scalatest:3.2.16`
import org.scalatest.{Filter => _, _}, flatspec._, matchers._
```

Algunas definiciones de tipos y funciones auxiliares

In [ ]:
```
object Signatures:
    abstract class List[A]:

        // Common HOFs
        def foldRight[B](nil: B)(cons: (A, B) => B): B
        def foldLeft[B](initial: B)(update: (B, A) => B): B
        def map[B](f: A => B): List[B]
        def flatMap[B](f: A => List[B]): List[B]
        def filter(f: A => Boolean): List[A]
        def forall(pred: A => Boolean): Boolean
        def exists(pred: A => Boolean): Boolean

        // Splits this list into a prefix/suffix pair at a given position.
        def splitAt(i: Int): Boolean

        // Returns new list with the elements in reversed order
        def reverse: List[A]

        // A copy of this sequence with an element inserted at the end.
        def appended(a: A): List[A]
```

In [ ]:
```
// type Tree[A] = 1 + Tree[A] * A * Tree[A]

enum Tree[A]:
    case Empty()
    case Node(left: Tree[A], root: A, right: Tree[A])

object Tree:

    def void[A]: Tree[A] =
        Empty()

    def leaf[A](a: A): Node[A] =
        Node(Empty(), a, Empty())

    def right[A](a: A, tree: Tree[A]): Node[A] =
        Node(Empty(), a, tree)

    def left[A](tree: Tree[A], a: A): Node[A] =
        Node(tree, a, Empty())

    def node[A](left: Tree[A], a: A, right: Tree[A]): Node[A] =
        Node(left, a, right)
```

```
import Tree._
```

Definiciones auxiliares sobre la correspondencia Curry-Howard y Tipos Algebraicos de Datos

	Function methods	Function values
Function1	`def f(a: A): B = ??? : B`	`val f: A => B = (a: A) => ??? : B`
Function2	`def f(a: A, b: B): C = ??? : C`	`val f: (A, B) => C = (a: A, b: B) => ??? : C`
Currified functions	`def f(a: A)(b: B): C = ??? : C`	`val f: A => B => C = (a: A) => (b: B) => ??? : C`
Generic functions	`def f[A, B](a: A): B = ??? : B`	`val f: [A, B] => A => B = [A, B] => (a: A) => ??? : B`

In [ ]:
```
trait Isomorphic[A, B]:

    def from(a: A): B

    def to(b: B): A

    // equality

    def equalA(a1: A, a2: A): Boolean =
        a1 == a2

    def equalB(b1: B, b2: B): Boolean =
        b1 == b2

    // Bijection laws

    def law1(a: A): Boolean =
        equalA(to(from(a)), a)

    def law2(b: B): Boolean =
        equalB(from(to(b)), b)
```

Type-driven development rules	Constructor	Destructor

(*exponents*)
 	
`A => B`
	`((a: A) => ??? : B): (A => B)`	`val f: A => B = …`
		`val a: A = …`
		`f(a): B`

(*products*)
 	
`(A, B)`
	`(??? : A, ??? : B): (A, B)`	`val t: (A, B) = …`
		`t._1 : A`
		`t._2 : B`

`(1)`
 	
`Unit`
	`(): Unit`	

(*sums*)
 	
`Either[A, B]`
	`Left(??? : A): Either[A, B]`
	
	`Right(??? : B): Either[A, B]`
		`val e: Either[A, B] = …`
		`(e match {`
		`  case Left(a: A) => … : C`
		`  case Right(b: B) => … : C`
		`}): C`

`(0)` `Nothing`		`val n: Nothing = …` `n: C`
`(var)` `A`		`val a: A = …` `a : A`

In [ ]:
```
type Not[P] = P => Nothing
type <=>[P, Q] = (P => Q, Q => P)
```

In [ ]:
```
trait ExcludedMiddle:
    def apply[P]: Either[P, Not[P]]
```

In [ ]:
```
trait DoubleNegation:
    def apply[P]: Not[Not[P]] => P
```

Ejercicio 1

(2 puntos)

a) (1 punto) Utiliza la correspondencia de Curry-Howard para demostrar la siguiente tautología de la lógica proposicional intuicionista:

$ \vdash (p \rightarrow q \rightarrow r) \rightarrow (s \rightarrow t \rightarrow p) \rightarrow s \rightarrow t \rightarrow (u \rightarrow v \rightarrow q) \rightarrow u \rightarrow v \rightarrow r$

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

b) (1 punto) Se desea utilizar la correspondencia de Curry-Howard para demostrar la validez de la siguiente tautología de la lógica clásica proposicional:

$ \vdash_c (\neg q \rightarrow \neg p) \rightarrow (\neg q \rightarrow p) \rightarrow q $

Para ello, utiliza como premisa adicional la ley de la doble negación aplicada a la proposición q.

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

Ejercicio 2

(1 punto)

Demuestra que, para cualesquiera tipos `A` y `B`, los tipos de datos `[C] => (A => C) => (B => C) => C` y `Either[A, B]` son isomorfos:

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

Ejercicio 3

(3,5 puntos)

La función `grouped` particiona una lista en subsecuencias consecutivas de un tamaño dado. Por ejemplo:

In [ ]:
```
class TestGrouped(grouped: (List[Int], Int) => List[List[Int]])
extends AnyFlatSpec with should.Matchers:

    "grouped" should "work" in:
        grouped(List(1,2,3,4,5), 1) shouldBe List(List(1), List(2), List(3), List(4), List(5))
        grouped(List(1,2,3,4,5,6), 2) shouldBe List(List(1, 2), List(3, 4), List(5,6))
        grouped(List(1,2,3,4,5,6,7,8,9), 3) shouldBe List(List(1, 2, 3), List(4, 5, 6), List(7, 8,
        grouped(List(1,2,3,4,5,6,7), 3) shouldBe List(List(1, 2, 3), List(4, 5, 6), List(7))
        grouped(List(1,2,3,4,5,6,7,8), 3) shouldBe List(List(1, 2, 3), List(4, 5, 6), List(7, 8))
        grouped(List(1,2,3,4,5), 5) shouldBe List(List(1, 2, 3, 4, 5))
        grouped(List(1,2,3,4,5), 6) shouldBe List(List(1, 2, 3, 4, 5))

    "grouped" should "equal standard implementation" in:
        (1 to 100).map: n =>
            (0 to 99).grouped(n).toList  == grouped((0 to 99).toList, n)
        .forall(identity) shouldBe true
```

Las implementaciones de los dos primeros apartados siguientes podrán hacer uso del método de la librería estándar de Scala `splitAt` perteneciente a la clase `List[A]`.

a) (1 punto) Implementa la función `grouped` mediante recursión (no final).

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

In [ ]:
```
run(new TestGrouped(grouped))
```

b) (1 punto) Implementa la función `grouped` mediante recursión final.

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

In [ ]:
```
run(new TestGrouped(grouped))
```

c) (1,5 puntos) Implementa la función `grouped` mediante la función `foldLeft`.

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

In [ ]:
```
run(new TestGrouped(grouped))
```

Ejercicio 4

(3,5 puntos)

La función `genFib` recibe un número positivo y devuelve la secuencia de números de Fibonacci menores que dicho número. Por ejemplo:

In [ ]:
```
class TestGenFib(genFib: Int => List[Int]) extends AnyFlatSpec with should.Matchers:

    "genFib" should "work" in:
        genFib(5) shouldBe List(0, 1, 1, 2, 3, 5)
```

```
        genFib(15) shouldBe List(0, 1, 1, 2, 3, 5, 8, 13)
        genFib(100) shouldBe List(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89)
        genFib(1000) shouldBe List(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987
```

Considérese ahora la función `genCollatz` que genera la secuencia de números Collatz para un número determinado. Por ejemplo:

In [ ]:
```
class TestGenCollatz(genCollatz: Int => List[Int]) extends AnyFlatSpec with should.Matchers:

    "genCollatz" should "work" in:
        genCollatz(5) shouldBe List(5, 16, 8, 4, 2, 1)
        genCollatz(11) shouldBe List(11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1)
        genCollatz(12) shouldBe List(12, 6, 3, 10, 5, 16, 8, 4, 2, 1)
        genCollatz(21) shouldBe List(21, 64, 32, 16, 8, 4, 2, 1)
```

Considérense las siguientes implementaciones de las funciones `genFib` y `genCollatz` basadas en un mismo patrón de diseño:

In [ ]:
```
def genFib(max: Int): List[Int] =
    def step(out: List[Int], next: (Int, Int)): List[Int] =
        if next._1 > max then out
        else
            step(next._1 :: out, (next._2, next._1 + next._2))

    step(Nil, (0,1)).reverse
```

In [ ]:
```
run(new TestGenFib(genFib))
```

In [ ]:
```
def genCollatz(n: Int): List[Int] =
    def step(out: List[Int], next: Int): List[Int] =
        if next == 1 then out
        else
            step(next :: out, if next % 2 == 0 then next / 2 else next * 3 + 1)

    (1 :: step(Nil, n)).reverse
```

In [ ]:
```
run(new TestGenCollatz(genCollatz))
```

a) (1,5 puntos) Implementa una función de orden superior, denominada `repeat`, que encapsule el patrón de diseño utilizado en las implementaciones de las funciones `genFib` y `genCollatz`.

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

b) (1,5 puntos) Implementa de una manera más modular las funciones `genFib` y `genCollatz` mediante la función `repeat`.

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA

def genFib(max: Int): List[Int] =
    ???
```

In [ ]:
```
run(new TestGenFib(genFib))
```

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA

def genCollatz(n: Int): List[Int] =
    ???
```

In [ ]:
```
run(new TestGenCollatz(genCollatz))
```

c) (0,5 puntos) Implementa de una manera más modular la función `grouped` del ejercicio anterior mediante la función `repeat`.

In [ ]:
```
// ESCRIBE AQUÍ TU RESPUESTA
```

In [ ]:
```
run(new TestGrouped(grouped))
```